PATRONES Y ESTILOS DE ARQUITECTURA DE SOFTWARE
ARQUITECTURA ORIENTADA A SERVICIOS (SoA)
Estilo de diseño donde las aplicaciones se construyen mediante la integración de servicios reutilizables e independientes comunicados a través de un bus de servicios (ESB).
CARACTERÍSTICA / VENTAJA
Ventaja: Facilita la reutilización de lógica empresarial a nivel corporativo e integra sistemas heterogéneos.
Desventaja: Suele ser complejo de implementar y el bus central de servicios puede convertirse en un cuello de botella.
CARACTERÍSTICA / VENTAJA
ventaja: permite escalar cada servicio de forma independiente y utilizar diferentes tecnologías según se necesiten.
Desventaja: Alta complejidad en la gestión, despliegue y monitoreo de la red y las comunicaciones.
ARQUITECTURA DE MICROSERVICIOS
Divide una aplicación en servicios pequeños e independientes que se comunican a través de la red y se despliegan por separado.
ARQUITECTURA EN CAPAS
Organiza el código en grupos horizontales donde cada capa cumple un rol específico (ej. presentación, negocio y datos). Cada capa solo se comunica con la inmediata inferior.
ARQUITECTURA CLIENTE - SERVIDOR
Descripción: Modelo distribuido donde los Clientes (como navegadores o apps) solicitan recursos y servicios, y los Servidores procesan esas peticiones y devuelven la respuesta.
CARACTERÍSTICA / VENTAJA
Organiza el código en grupos horizontales donde cada capa cumple un rol específico (ej. presentación, negocio y datos). Cada capa solo se comunica con la inmediata inferior.
CARACTERÍSTICA / VENTAJA
Descripción: Permite centralizar la información y la lógica pesada en el servidor, facilitando el respaldo de datos y permitiendo que múltiples clientes accedan simultáneamente.
ARQUITECTURA MONOLÍTICA
Descripción: Estilo tradicional donde todas las partes de una aplicación (interfaz, lógica de negocio y acceso a datos) se combinan en un único programa y se despliegan juntas.
ARQUITECTURA MVC
Descripción: Patrón de diseño que separa los datos (Modelo), la interfaz de usuario (Vista) y la lógica que interconecta a ambos (Controlador).
CARACTERÍSTICA / VENTAJA
Ideal para proyectos pequeños o fases iniciales. Su puesta en marcha es directa porque no requiere la complejidad de coordinar múltiples servicios independientes.
CARACTERÍSTICA / VENTAJA
Descripción: Optimiza el desarrollo web permitiendo modificar la apariencia visual sin alterar la lógica de negocio, mejorando la organización y escalabilidad del código.
ARQUITECTURA HEXAGONAL
Organiza el sistema aislando el núcleo de la lógica de negocio de los agentes externos (Bases de Datos, UI, APIs) mediante puertos y adaptadores.
CARACTERÍSTICA / VENTAJA
Ventaja: El núcleo de la aplicación resulta altamente testeable, independiente de librerías externas y fácil de mantener.
Desventaja: Requiere crear múltiples capas de abstracción y código adicional (boilerplate), aumentando la complejidad inicial.